

Diseño y Análisis de Algoritmos

Ayudantía 3: Correctitud: Invariante de Loop

Ayud. Ferran Rojas Andreu

1. Algoritmo

INSERTION-SORT(A)

```
1  for  $i = 1$  to  $n - 1$ 
2      key =  $A[i]$ 
3       $j = i - 1$ 
4      while  $j \geq 0$  and  $A[j] > \mathbf{key}$ 
5           $A[j + 1] = A[j]$ 
6           $j = j - 1$ 
7       $A[j + 1] = \mathbf{key}$ 
8  return  $A$ 
```

2. Invariante de loop

Invariante $I(i)$. Al inicio de la iteración i del bucle **for** exterior (con $1 \leq i \leq n - 1$):

el subarreglo $A[0 \dots i - 1]$ contiene los elementos originales de $A_0[0 \dots i - 1]$ en orden no decreciente.

En palabras: el prefijo ya procesado quedó ordenado y el algoritmo nunca pierde ni duplica elementos (solo los reordena).

3. (a) Inicialización

Antes de la primera iteración, $i = 1$. El subarreglo $A[0 \dots 0]$ tiene un único elemento, por lo que está trivialmente ordenado en orden no decreciente y contiene el elemento original $A_0[0]$. Luego $I(1)$ se cumple.

4. (b) Mantenimiento

Suponga que $I(i)$ es verdadero al inicio de la iteración i , es decir, $A[0 \dots i - 1]$ está ordenado y contiene los elementos originales de $A_0[0 \dots i - 1]$.

Al comienzo de la iteración se guarda $\mathbf{key} = A[i]$ y se ejecuta el bucle **while**:

- Mientras $j \geq 0$ y $A[j] > \mathbf{key}$, se desplaza una posición a la derecha el elemento $A[j]$ (asignación $A[j + 1] \leftarrow A[j]$). Este desplazamiento solo reubica elementos mayores que $\mathbf{key}$ dentro del prefijo $A[0 \dots i]$, sin alterar el conjunto de elementos.
- El bucle se detiene cuando $j = -1$ o bien $A[j] \leq \mathbf{key}$. En ese momento la posición $j + 1$ está libre, y la asignación $A[j + 1] \leftarrow \mathbf{key}$ inserta $\mathbf{key}$ en su lugar correcto.

Por lo tanto, al finalizar la iteración, el nuevo prefijo $A[0 \dots i]$:

- contiene exactamente los elementos originales de $A_0[0 \dots i]$ (los mismos i elementos del prefijo más **key**, que era $A_0[i]$), y
- está en orden no decreciente: todos los elementos $A[0..j] \leq \mathbf{key}$ y todos los desplazados $\mathbf{key} < A[j+1] \leq \dots \leq A[i]$.

Con esto, al inicio de la siguiente iteración se cumple $I(i+1)$.

5. (c) Terminación

El bucle **for** exterior recorre exactamente $i = 1, 2, \dots, n-1$, por lo que siempre termina. Al finalizar, se cumple $I(n)$, que establece:

$A[0 \dots n-1]$ contiene los elementos originales del arreglo completo A_0 , en orden no decreciente.

Como el algoritmo retorna A , la salida es correcta. ■

6. Ejemplo ilustrativo

Para $A = \langle 5, 2, 4, 1 \rangle$:

i	key	A después de la iteración
1	2	$\langle 2, 5, 4, 1 \rangle$
2	4	$\langle 2, 4, 5, 1 \rangle$
3	1	$\langle 1, 2, 4, 5 \rangle$

En cada fila, el prefijo $A[0 \dots i]$ queda ordenado, tal como afirma el invariante.

Conclusión. Por inicialización, mantenimiento y terminación, INSERTION-SORT ordena correctamente todo arreglo de entrada:

INSERTION-SORT(A) retorna A ordenado en orden no decreciente .

7. (a) Ejecución sobre $A = \langle 2, 1, 5, 1, 3, 2 \rangle$, $k = 3$

Ventana inicial: $\text{window} = 2 + 1 + 5 = 8$, $\text{best} = 8$.

i	window antes	operación	best
—	—	inicial: 8	8
4	8	$8 - A[1] + A[4] = 8 - 2 + 1 = 7$	8
5	7	$7 - A[2] + A[5] = 7 - 1 + 3 = 9$	9
6	9	$9 - A[3] + A[6] = 9 - 5 + 2 = 6$	9

El algoritmo retorna

$$\boxed{\text{best} = 9}.$$

Verificación: las ventanas de tamaño 3 son $\langle 2, 1, 5 \rangle = 8$, $\langle 1, 5, 1 \rangle = 7$, $\langle 5, 1, 3 \rangle = 9$, $\langle 1, 3, 2 \rangle = 6$. El máximo es 9. ✓

8. (b) Demostración por invariante de loop

Invariante $I(i)$. Al inicio de la iteración i del bucle **for** (con $k + 1 \leq i \leq n + 1$):

(1) **window** es la suma del subarreglo $A[i - k \dots i - 1]$ (la ventana de largo k que termina en $i - 1$);

(2) **best** es el máximo de las sumas de todas las ventanas de largo k contenidas en $A[1 \dots i - 1]$.

Inicialización. Antes de la primera iteración, $i = k + 1$. Por la asignación inicial, **window** = $\sum_{t=1}^k A[t]$, que es la ventana que termina en $k = i - 1$. Además **best** = **window** es la suma de la única ventana contenida en $A[1..k]$. Luego $I(k + 1)$ se cumple.

Mantenimiento. Suponga $I(i)$ verdadero al inicio de la iteración i . La asignación

$$\text{window} \leftarrow \text{window} - A[i - k] + A[i]$$

saca el primer elemento de la ventana anterior ($A[i - k]$) e incorpora $A[i]$; por tanto, al finalizar la asignación, **window** es la suma de $A[i - k + 1 \dots i]$, la ventana que termina en i . La asignación **best** $\leftarrow \max(\text{best}, \text{window})$ incorpora esta nueva ventana al máximo. Así, al inicio de la siguiente iteración, $I(i + 1)$ se cumple.

Terminación. El bucle recorre exactamente $i = k + 1, \dots, n$, por lo que termina con $i = n + 1$. Por $I(n + 1)$, **best** es el máximo de las sumas de todas las ventanas de largo k contenidas en $A[1..n]$, que es exactamente la respuesta pedida. Como el algoritmo retorna **best**, la salida es correcta. ■

9. (c) Complejidad

La suma inicial cuesta $O(k)$. El bucle recorre $n - k$ iteraciones, cada una con trabajo constante $O(1)$. Por lo tanto:

$$\boxed{\text{Tiempo } O(n), \quad \text{Espacio adicional } O(1)}.$$

Idea clave. En vez de recomputar la suma de cada ventana desde cero (coste $O(k)$ por ventana, $O(nk)$ total), la ventana deslizante reutiliza la suma anterior actualizándola con una resta y una suma, logrando tiempo lineal.